Beyond Compression Ratio: Evaluating the Algorithmic and Systems Utility of Long-Context KV-Cache Compression*

Anonymous authors
Paper under double-blind review

Abstract

Key-value (KV) cache compression is almost always reported as a compression ratio paired with a task score. We show that this pairing is not sufficient to predict how a method will behave in a serving system, and that the two numbers do not even predict each other. Across 3,150 paired query executions spanning seven compression methods, three open-weight models (Gemma 4 E2B, Gemma 4 E4B, Qwen2.5 7B), five long-context tasks and context lengths from 2K to 128K tokens, we find that (i) at an *identical* nominal budget of 4 bits per KV element, task accuracy ranges from 2.5% to 47.3% on Gemma 4 E2B — an 18.7× spread across methods that a compression ratio cannot distinguish; (ii) the memory a method saves carries essentially no information about the hardware efficiency it realises (Spearman $\rho = +0.04$, $p = 0.84$ over 36 operating points), and the compressed footprint carries none about prefill cost ($\rho = +0.01$, $p = 0.98$); and (iii) two methods matched to within 0.55 bits per element and 2.0 accuracy points differ by 30× in time-to-first-token and 3.5× in peak VRAM on the same model and prompts. Representation efficiency, task utility and serving efficiency are three separate axes, and a benchmark that collapses them hides exactly the information a practitioner needs.

We introduce CRBench, the measurement instrument these findings require: a query-level primitive that pairs every compressed execution with an uncompressed dense execution on the identical prompt, an effective-bits-per-element accounting that charges each representation for its mandatory metadata, and a decoupled two-track report separating algorithmic representation utility ($\mathcal{S}_{\text{res}}$) from measured serving utility ($\mathcal{S}_{\text{sys}}$). Charging for metadata is not a formality: it raises a factorised representation from a nominal 5.00 to a realised 6.51 bits/elem and a nominal 4-bit quantiser from 4.05 to 4.55 bits/elem, understatements of 30% and 12% that reorder the resulting table. We report per-method provenance, state which memory savings are physically realised and which are analytically accounted, and scope every conclusion to the models, implementations and hardware evaluated. Code, raw per-query records and the scripts that regenerate every number and figure are released.

1 Introduction

Long-context inference is bounded by the key-value (KV) cache. Because standard attention caches key and value tensors for every past token at every layer and KV head, cache memory grows linearly in sequence length L and batch size B:

$$M_{\text{KV}} = 2\, N_{\text{layers}}\, N_{\text{kv heads}}\, d_{\text{head}}\, L\, B \cdot \text{sizeof(dtype)} \quad \text{[bytes]}. \tag{1}$$

For a Llama-3.1-8B-shaped model (32 layers, 8 KV heads under grouped-query attention, head dimension 128, 16-bit) a single 128K-token sequence needs 16.0 GiB for the cache alone — more than the weights, and

*Large language models were used as assistive tools for stylistic editing and drafting under the authors' direct supervision. The authors take full responsibility for all content, experimental methodology, analysis, and scientific claims.

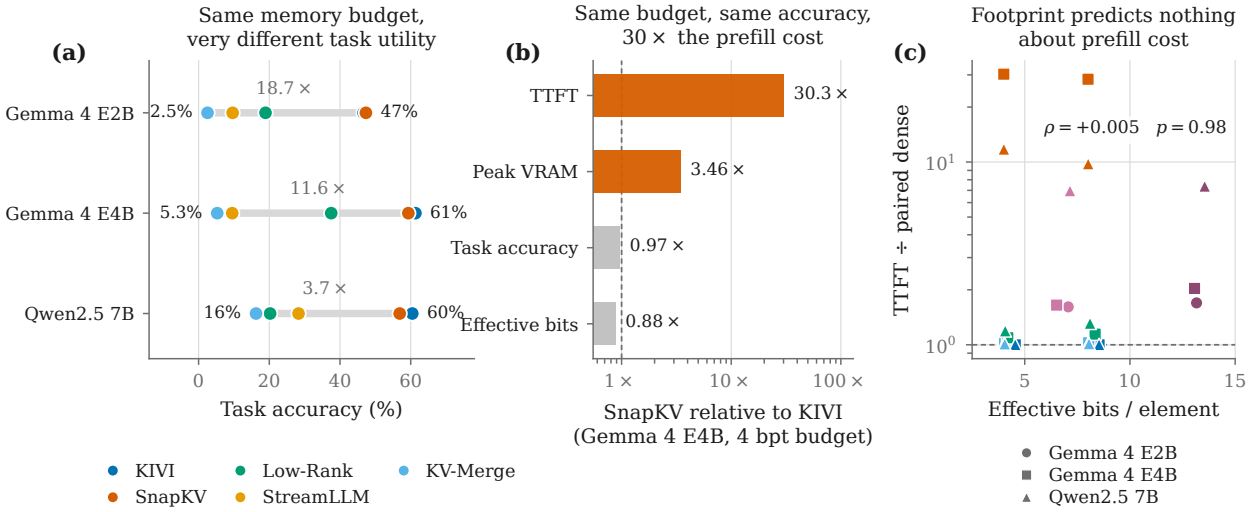


Figure 1: **Three axes that do not substitute for one another. (a)** Five methods held at an identical nominal budget of 4 bits per KV element — realised payloads within 0.19 bits/elem of one another, effective footprints within 0.55 once metadata is charged — differ in task accuracy by up to 18.7×. A compression ratio cannot tell these methods apart; only task measurement can. **(b)** SnapKV and KIVI on Gemma 4 E4B agree to within 0.55 bits per element and 2.0 accuracy points, and then diverge by 3.5× in peak VRAM and 30× in time-to-first-token. Grey bars are the representation and quality axes; orange bars are the serving axis. **(c)** Over all 36 compressed operating points, the effective footprint of a representation is uninformative about its prefill cost. Marker shape denotes model, colour denotes method. Panel (b) reflects the SnapKV integration described in §5.5, not SnapKV's fused reference kernel.

more than most consumer GPUs have. A large literature answers this with quantisation, token eviction, pooling, low-rank projection and factorised state (Liu et al., 2024; Li et al., 2024; Xiao et al., 2023; Zhang et al., 2023; Ge et al., 2023; Hooper et al., 2024).

The reporting convention, and why it fails. These methods are almost universally summarised by two numbers: a compression ratio (or a nominal bit-width) and a task score. The implicit promise is that the first number tells you what the method costs and the second tells you what it costs you. Our measurements say otherwise, in three distinct ways.

First, the compression ratio does not determine task utility. Figure 1(a) holds five methods at the same nominal 4-bit budget — their realised payloads agree to within 0.19 bits per element, and their effective footprints to within 0.55 — and finds accuracy spanning 2.5% to 47.3% on Gemma 4 E2B, 5.3% to 61.3% on Gemma 4 E4B, and 16.2% to 60.4% on Qwen2.5 7B. Pooled over all 36 compressed operating points in our study, the rank correlation between memory saved and accuracy achieved is $\rho = -0.42$ ($p = 0.010$): saving more memory is, if anything, associated with retaining *less* task ability. A compression ratio reported without a task measurement is close to uninformative.

Second, the task score does not determine serving behaviour. Figure 1(b) takes two methods that agree on both of the reported numbers — within 0.55 bits per element and 2.0 accuracy points on the same model and the same prompts — and shows them diverging by 3.46× in peak VRAM and 30.3× in time-to-first-token (TTFT). Nothing in the conventional report anticipates this. Over all operating points, memory saved has no rank relationship to the measured hardware efficiency multiplier ($\rho = +0.04$, $p = 0.84$), and the effective footprint has none to prefill cost ($\rho = +0.01$, $p = 0.98$) or to measured peak VRAM ($\rho = -0.08$, $p = 0.63$).

Third, the compression ratio is usually misreported. A representation's real cost is its payload *plus* the metadata that makes the payload decodable: quantisation scales and zero points, codebooks, subspace bases, eviction index tables. Charging for that metadata moves a factorised representation from a nominal 5.00 to a realised 6.51 bits per element, and a nominal 4-bit quantiser from 4.05 to 4.55 — understatements

of 30% and 12%, in the direction that flatters the method. Because the overhead is concentrated in particular method families rather than spread evenly, correcting it also reorders the ranking.

What follows. The rest of the paper takes these three findings seriously as a measurement problem. §2 states the three axes precisely and argues why none is recoverable from the others. §4 develops CRBench, the instrument that measures them jointly: a query-level primitive that runs every compressed configuration against a dense reference on the identical prompt, an effective-bits accounting that charges for metadata, and two reported scores that are deliberately not merged into one. §5 and §6 give the study and its results, organised around the three questions above. §7 draws out what this means for method authors and practitioners, and §8 is explicit about what our evidence does and does not support.

Contributions.

- **An empirical separation of three axes that the field reports as two.** At matched nominal bitrates, task accuracy varies by up to 18.7×; at matched bitrate *and* matched accuracy, serving cost varies by 30×. We quantify the (near-zero) rank association between representation footprint and realised hardware behaviour across 36 operating points and three models.
- **Effective bits per element (b_{eff}), an accounting that charges for metadata.** We show it is not a bookkeeping detail: it changes reported bitrates by 6–28% depending on the method family, and it penalises exactly the methods that carry auxiliary state. Applied honestly, it demotes configurations we might otherwise have highlighted.
- **A decoupled two-track report ($\mathcal{S}_{\text{res}}$, $\mathcal{S}_{\text{sys}}$) with an auditable dense pairing.** Every query is executed twice on the identical prompt, once dense and once compressed, and both outcomes are logged; coverage is reported alongside the scores rather than folded into them.
- **A study designed for interpretability rather than a leaderboard.** 3,150 paired executions over 75 distinct queries per model, at two controlled bit budgets, with per-method provenance stating which implementation was used and whether its savings were physically realised or analytically accounted.

2 Three Axes, and Why They Do Not Substitute

We distinguish three properties of a KV-cache compression method that the literature routinely conflates.

Axis 1: representation efficiency. How many bits of state the method actually keeps per KV element, including everything needed to decode that state. This is a property of the representation alone. It is what a compression ratio is trying to express, and it is what determines how much longer a context fits in a fixed memory budget.

Axis 2: task utility. How much of the model's task ability survives the transform, measured on the same prompts against the same dense reference. This is a property of the representation *and* the model *and* the task.

Axis 3: serving efficiency. What the method costs in wall-clock prefill latency, decode throughput and resident device memory when actually executed. This is a property of the representation, its *implementation*, and the hardware.

Why Axis 1 does not give Axis 2. Two representations can retain the same number of bits and spend them completely differently. A 4-bit quantiser spends its budget uniformly across all tokens; an eviction policy spends 16 bits on a quarter of the tokens and nothing on the rest; a pooling scheme spends them on averages that may not correspond to any token the model needs. Which allocation is right depends on where the task's information lies, which the bit count cannot express. §6.1 measures the resulting spread.

Why Axis 1 does not give Axis 3, in four concrete ways.

1. **Metadata is not free and is not proportional.** Group-wise quantisation must store a scale and zero point per group; a factorised scheme must store anchors and subspace bases; an eviction method needs only a position index. At a nominal 4-bit budget these carry overheads of 11%, 23% and 0.1% of realised footprint respectively in our measurements (§6.2). A ratio computed on payload alone therefore mis-ranks them.
2. **Prefill cost is decided by the transform, not the footprint.** A method that reduces resident memory can still add an expensive pass over the prompt. Attention-score pooling, SVD and block factorisation all do work proportional to the *uncompressed* prompt before any saving materialises. Our measured TTFT inflation ranges from 0.96× to 30.3× dense at effectively identical footprints.
3. **Implementation efficiency is part of the answer.** The same algorithm, expressed as a fused kernel or as un-fused tensor operations, is a different serving proposition. This is not a defect of the measurement — it is the quantity a practitioner is actually choosing between, and it cannot be read off a paper's compression ratio.
4. **Peak memory is not resident memory.** A transform that ends up smaller can pass through a much larger intermediate. In our runs the method with the smallest analytic footprint at 4 bits also had the largest measured peak VRAM, 3.5× the alternative.

The practical consequence is that the three axes must be measured, separately, on the same queries. That is what CRBench is for.

3 Related Work

Long-context evaluation. Needle-in-a-Haystack (Kamradt, 2023), RULER (Hsieh et al., 2024), LongBench (Bai et al., 2023b; 2024), L-Eval (An et al., 2023), BAMBOO (Dong et al., 2023) and ∞-Bench (Zhang et al., 2024b) measure what a model can do with a long context. They do not track what that context costs, and they evaluate uncompressed models, so they cannot separate a representation's contribution from the base model's.

Compression methods. KIVI (Liu et al., 2024), KVQuant (Hooper et al., 2024) and LLM.int8() (Dettmers et al., 2022) quantise; SnapKV (Li et al., 2024), StreamingLLM (Xiao et al., 2023), H_2O (Zhang et al., 2023) and Scissorhands (Liu et al., 2023) evict; Ge et al. (2023) and PyramidKV (Zhang et al., 2024a) merge or allocate adaptively; low-rank approaches project (Sharma et al., 2024). Each reports on its own task subset, at its own bit accounting, on its own hardware, typically against a baseline reproduced by its own authors. The literature is therefore rich in individually credible results that cannot be placed on a common axis.

Systems measurement. Serving stacks report latency, throughput and memory but not task quality, and generally not per representation. The gap we address is not that any one of these communities measures badly; it is that the three measurements are never taken on the same queries, so their disagreements are invisible.

4 CRBench: Measuring the Three Axes Jointly

4.1 The paired query primitive

Let $\mathcal{M}$ be an autoregressive model with fixed weights, $\mathcal{D} = \{(x_i, y_i, \tau_i, L_i)\}_{i=1}^{N}$ a dataset of queries with targets, task types and context lengths, $\mathcal{R}_{\text{dense}}$ the uncompressed 16-bit reference representation, and $\mathcal{A}(\mathcal{B})$ a candidate representation under budget $\mathcal{B}$.

Definition 1 (Paired evaluation unit)**.** The atomic unit of CRBench is the 4-tuple $\mathcal{U}_i = (\mathcal{M}, (x_i, y_i), \mathcal{R}_{\text{dense}}, \mathcal{A}(\mathcal{B}))$. For each unit CRBench executes the dense reference $\hat{y}_{i,\text{dense}} = \mathcal{M}(x_i \mid \mathcal{R}_{\text{dense}})$

Table 1: What existing evaluation practice records. "Dense anchor" means an uncompressed run of the *same* model is reported; "paired" means that run is on the *same* query. △ = sometimes, inconsistently.

Evaluation practice	Task quality	Memory	Latency	Dense anchor	Paired
Long-context suites (LongBench, L-Eval)	✓	✗	✗	✗	✗
Synthetic probes (NIAH, RULER)	✓	✗	✗	✗	✗
Serving-system profilers	✗	✓	✓	✗	✗
Compression method papers	✓	△	△	△	✗
CRBench (this work)	✓	✓	✓	✓	✓

and the candidate $\hat{y}_{i,\text{method}} = \mathcal{M}(x_i \mid \mathcal{A}(\mathcal{B}))$ on the identical prompt, and records for both: the task score $s \in [0, 1]$, the analytic state footprint in bytes, and the measured TTFT, decode throughput and peak device memory.

Pairing at the query level rather than the dataset level is what makes the three axes comparable. Every ratio we report — accuracy retention, TTFT inflation, VRAM inflation — is formed against a dense execution of the same prompt in the same session, so it cannot be contaminated by a differently configured or differently reported baseline.

4.2 Task quality

Definition 2 (Absolute task quality)**.** For a completed candidate run with evaluator score $s_{i,\text{method}} \in [0, 1]$,

$$Q_{\text{abs},i} = 100 \cdot \min\big(1, \max(0,\ s_{i,\text{method}})\big). \tag{2}$$

We deliberately report absolute success rather than retention relative to dense. Retention rewards a method for being paired with a weak baseline: a method that scores 20% where dense scores 20% has 100% retention and is useless. The paired dense outcome is still recorded for every query — it defines coverage, enables failure diagnosis, and lets a reader recompute retention where a task has a defensible chance floor — but it is not substituted into the headline number. We log the signed paired difference $\Delta_i = s_{i,\text{method}} - s_{i,\text{dense}}$ throughout.

4.3 Memory accounting: effective bits per element

This is the axis most often misreported, so we make the accounting explicit. For any representation the total state is

$$M_{\text{method}}(L) = \underbrace{M_{\text{algo}}(L)}_{\text{compressed payload}} + \underbrace{M_{\text{meta}}(L)}_{\text{scales, zero points, codebooks, bases, index tables}} + \underbrace{M_{\text{align}}(L)}_{\text{padding}} \quad \text{[bytes]}. \tag{3}$$

Definition 3 (Effective bits per element)**.** Normalising the *total* state by the uncompressed KV element count,

$$b_{\text{eff}} = \frac{8 \cdot M_{\text{method}}(L)}{2\, N_{\text{layers}}\, N_{\text{kv heads}}\, d_{\text{head}}\, L} \quad \text{[bits/element]}, \tag{4}$$

which is exactly 16.0 for an uncompressed FP16/BF16 cache.

Why the metadata term is load-bearing. A method's nominal bit-width describes only M_{algo}. The remaining terms are not optional: without the scales, the quantised payload cannot be dequantised; without the bases, the projection cannot be inverted. They are also not uniform across method families — group-wise quantisation pays per group, a subspace method pays per block, and an eviction method pays only a token index — so omitting them does not merely shift every method equally, it *re-ranks* them. §6.2 quantifies this at 6–28% of realised footprint depending on the method.

Definition 4 (Memory saving)**.** $R_{\text{mem},i} = 100 \cdot \max\big(0,\ 1 - M_{i,\text{method}}/M_{i,\text{dense}}\big) \in [0, 100]$.

Analytic versus physically realised savings. We distinguish two things a compression result can mean. A saving is *physically realised* when the resident cache tensors actually shrink, and *analytically accounted* when the scheme's footprint is computed from its definition while the harness keeps the transformed values in a wider dtype for exactness. Both are legitimate — the first measures a system, the second measures a representation — but they are not interchangeable, and conflating them is how a paper ends up claiming a serving win it did not measure. §5.5 states which is which for every method here, and peak-VRAM measurements in the Part 2 track always reflect what was physically resident.

4.4 Part 1: algorithmic representation utility

$$\mathcal{S}_{\text{res}} = \alpha\, Q_{\text{abs}} + (1-\alpha)\, R_{\text{mem}}, \qquad \alpha = 0.70 \text{ by default.} \tag{5}$$

The linear form is chosen for transparency, not optimality: it is monotone in both arguments, bounded on $[0, 100]$, Pareto consistent, and — unlike a product or harmonic form — never lets one dimension annihilate the other, so a reader can always decompose a score back into the two quantities that produced it (proof in Appendix A; comparison with multiplicative alternatives in Appendix B). At $\alpha = 0.70$ one accuracy point is worth 2.33 points of memory saving. We report α sensitivity in Figure 8 rather than asking the reader to accept the default; the qualitative ordering is stable over $\alpha \in [0.4, 0.9]$.

Aggregation over a dataset uses the arithmetic mean of Q_{abs} and a byte-weighted aggregate of memory, $\bar{R}_{\text{mem}} = 100(1 - \sum_i M_{i,\text{method}} / \sum_i M_{i,\text{dense}})$, so that long contexts — where compression matters — are not diluted by short ones. Failed queries are charged the dense footprint and score zero rather than being dropped.

Coverage is reported, not folded in. We report paired coverage $C_{\text{paired}} = N_{\text{paired success}}/N_{\text{total}}$ beside $\mathcal{S}_{\text{res}}$ as a pair $(\mathcal{S}_{\text{res}}, C_{\text{paired}})$ rather than multiplying it in, so that a run that failed for allocator reasons is visibly distinguishable from a run that completed and scored zero. Coverage was 1.00 for every configuration reported here.

4.5 Part 2: measured serving utility

Part 2 records TTFT (wall-clock to first emitted token, including all transform work), decode throughput $\mathcal{T}_{\text{dec}}$ in tokens/s, and peak resident device memory $\mathcal{M}_{\text{peak}}$. Each is converted to a paired efficiency against the *same query's* dense execution:

$$\phi_{\text{ttft}} = \min\Big(1, \tfrac{\text{TTFT}_{\text{dense}}}{\text{TTFT}_{\text{method}}}\Big), \quad \phi_{\text{thru}} = \min\Big(1, \tfrac{\mathcal{T}_{\text{method}}}{\mathcal{T}_{\text{dense}}}\Big), \quad \phi_{\text{vram}} = \min\Big(1, \tfrac{\mathcal{M}_{\text{dense}}}{\mathcal{M}_{\text{method}}}\Big), \tag{6}$$

combined into a hardware multiplier and a system score

$$\phi = \phi_{\text{ttft}}^{0.35}\, \phi_{\text{thru}}^{0.35}\, \phi_{\text{vram}}^{0.30}, \qquad R_{\text{sys}} = 100\phi, \qquad \mathcal{S}_{\text{sys}} = \alpha \mathcal{S}_{\text{res}} + (1-\alpha) R_{\text{sys}}. \tag{7}$$

The weights give interactive responsiveness (TTFT and decode, 0.35 each) slightly more emphasis than resident footprint (0.30); we make no claim that this is the only defensible weighting, and the components are always reported separately so a reader can reweight. ϕ is computed per query and then averaged; the TTFT, throughput and VRAM values in our tables are descriptive means and are not substituted back into Equation 7.

$\mathcal{S}_{\text{sys}}$ is a summary, not evidence. Because $\mathcal{S}_{\text{sys}}$ contains $\mathcal{S}_{\text{res}}$ by construction, a correlation between them is circular and we never use one. Where we ask whether algorithmic utility predicts serving behaviour we correlate against ϕ, the pure hardware term.

5 Experimental Setup

5.1 Design

Each model is evaluated on 5 tasks × 5 context lengths × 3 samples = 75 distinct queries. Each query is executed under 14 configurations: five fixed-budget methods at nominal budgets of 4 and 8 bits per element,

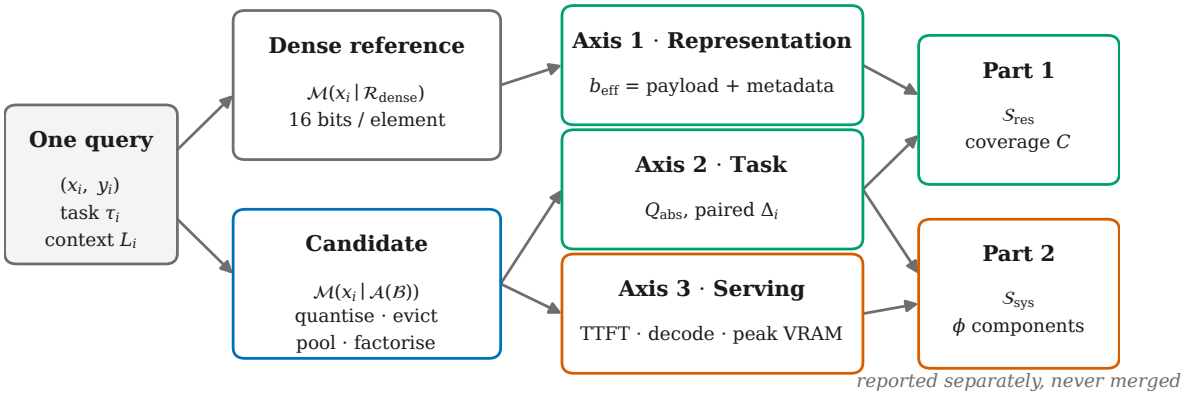


Figure 2: **CRBench measurement flow.** A single query is executed twice on the identical prompt — once under the dense reference, once under the candidate representation — and the two executions are compared on all three axes. Part 1 ($\mathcal{S}_{\text{res}}$) reports the representation; Part 2 ($\mathcal{S}_{\text{sys}}$) reports what the representation cost on the hardware it ran on. The tracks are reported separately by design.

two adaptive DKV configurations, and the dense reference (recorded in both budget sweeps). This gives 1,050 paired executions per model and 3,150 in total. Decoding is greedy ($T = 0$); the generation budget is fixed per task (32–256 new tokens) and identical across every configuration, so no method is advantaged by a longer or shorter budget.

Reporting operating points separately, rather than pooling the two budget sweeps into one row per method, is deliberate: a pooled "≈6 bits" row averages two configurations whose accuracies can differ by more than 30 points, and it is precisely the matched-budget comparison that carries the paper's evidence.

5.2 Models and context ladders

- **Gemma 4 E2B** (`google/gemma-4-e2b-it`): 2K, 8K, 32K, 64K, 128K.
- **Gemma 4 E4B** (`google/gemma-4-e4b-it`): 2K, 4K, 8K, 16K, 32K.
- **Qwen2.5 7B** (`Qwen/Qwen2.5-7B-Instruct`): 2K, 4K, 8K, 16K, 32K.

5.3 Tasks

Single-needle and multi-needle retrieval at parameterised depths; RULER key–value lookup among distractors (Hsieh et al., 2024); RULER multi-step variable tracking (Hsieh et al., 2024); and synthetic cross-document multi-hop QA. Two of the five behave as controls rather than discriminators in our study and we treat them as such in §6.5.

5.4 Environment

All runs were executed on a single NVIDIA GeForce RTX 4070 SUPER (12 GB VRAM), with PyTorch 2.6.0 (CUDA 12.4 build), `transformers` 5.15.1 and Python 3.12.10 on Windows 11. Model weights were loaded in 4-bit NormalFloat (NF4) via bitsandbytes so that the weight footprint is held constant and the 128K ladder fits in 12 GB; the KV cache under study is unaffected by this choice. Prefill used chunked forward passes (4,096 tokens) to avoid intermediate allocation spikes. Because every comparison is paired against a dense execution on the same device in the same session, absolute latencies are hardware-specific but the reported ratios are not distorted by machine-to-machine variation.

Determinism. The dense reference was executed twice for every query, once in each budget sweep. On Gemma 4 E4B and Qwen2.5 7B all 150 re-executions produced byte-identical outputs. On Gemma 4 E2B,

Table 2: **Per-method provenance.** "Upstream" means the authors' own released code was invoked; "CR-Bench" means a reference implementation written for this harness following the published scheme. "Realised" means the resident cache tensors physically shrank; "analytic" means the footprint was computed from the scheme while values were retained in a wider dtype, so quality is exact for the scheme but no device memory was freed.

Method	Impl.	Saving	Budget	Notes
KIVI (INT4/INT8)	CRBench	analytic	4, 8	Asymmetric grouped min–max, per-channel keys and per-token values, following Liu et al. (2024). Partial key groups stay at 16 bits and are charged as such.
SnapKV	Upstream	realised	4, 8	`SnapKVCluster.update_kv` at `e216ddc`. Upstream's monkeypatch targets `transformers` 4.37 and Llama/Mistral only, so under `transformers` 5.x the cluster is driven directly, *outside* the fused attention path.
StreamingLLM	Upstream	realised	4, 8	`StartRecentKVCache` at `2e50426`; policy unmodified.
KV-Merging	CRBench	realised	4, 8	Temporal mean-pooling of consecutive states, sinks and recent window preserved (Ge et al., 2023).
Low-Rank KV	CRBench	analytic	4, 8	Per-head mean-centred truncated SVD (Sharma et al., 2024); footprint is the U, Σ, V factor size.
DKV-Mid / High	Upstream	analytic	adaptive	Anchor-plus-low-rank differential compression. The block compressor is driven over the resident cache; the repository's paged serving runtime and Triton decode kernel were unavailable on this platform, so these numbers characterise the representation, *not* that runtime's decode speedup.
Dense FP16	—	—	16	Uncompressed reference.

64/75 produced identical text and exactly one query changed score — a multi-needle prompt at 128K. Run-to-run variance is therefore negligible at the resolution of our claims, and is smallest where the claims are strongest.

5.5 Methods and provenance

Table 2 states, for every method, which implementation produced the numbers and whether its memory saving was physically realised or analytically accounted. We consider this disclosure part of the result rather than an appendix detail: a serving claim is only as strong as the code path that produced it.

On SnapKV in particular. The prefill cost we measure for SnapKV is a property of this integration, not of SnapKV as published: driven outside its fused attention path, the observation-window pooling runs as separate tensor operations over the full prompt. We report it because it is the honest measurement of the code path that was runnable here, and because it illustrates the paper's own thesis — serving efficiency is a property of an algorithm *and* its implementation, and is therefore not predictable from the algorithm's compression ratio. It should not be read as a claim about SnapKV's achievable latency. The same caveat, in the opposite direction, applies to DKV: its representation is measured, its serving runtime is not.

6 Results

6.1 Does the compression ratio predict task utility? No.

Table 3 reports every operating point. Reading across a fixed nominal budget isolates Axis 1 from Axis 2: the five fixed-budget methods at 4 bits realise payloads within 0.19 bits per element of one another, and effective footprints within 0.55 once metadata is charged, so any accuracy difference between them is attributable to *how* they spend the budget rather than how much they spend.

The differences are large. On Gemma 4 E2B accuracy at 4 bits ranges from 2.5% (KV-Merging) to 47.3% (SnapKV), a factor of 18.7; on Gemma 4 E4B from 5.3% to 61.3% (11.6×); on Qwen2.5 7B from 16.2% to 60.4% (3.7×). At 8 bits the spreads narrow but remain substantial (22.2, 21.2 and 37.4 accuracy points respectively), because the methods that fail at 4 bits recover unevenly: StreamingLLM gains 26.5 points on E2B between the two budgets while KIVI gains none (46.7% → 46.2%, within noise). How much accuracy a bit buys is itself method-dependent.

Pooled over all 36 compressed operating points, the rank correlation between memory saved and accuracy is $\rho = -0.42$ ($p = 0.010$); per model it is -0.28, -0.64 and -0.39. The negative sign is unsurprising — within a method family, saving more costs accuracy — but the point is the absence of any positive predictive relationship *across* methods. A reader given only a compression ratio cannot order these methods on task ability, even approximately.

6.2 How much does metadata cost? Six to twenty-eight percent.

Figure 3 decomposes each operating point into payload and mandatory metadata. The overhead is heavily method-dependent. Eviction and pooling methods store only a token index and pay 0.1–0.4%. Grouped quantisation pays for scales and zero points: a nominal 4-bit configuration realises 4.51–4.55 bits per element (11% overhead), and because the metadata is per group rather than per bit, the same scheme at 8 bits pays only 5.8–5.9%. The factorised DKV representations pay most: 18–23%, moving DKV-Mid from a nominal 5.00 to a realised 6.51 bits per element on Gemma 4 E4B, and DKV-High from 10.68 to 13.07 — at which point it is saving 18% of the dense footprint rather than the 33% a payload-only accounting would report.

This matters for ranking, not only for absolute numbers. Because the overhead is concentrated in one method family, charging for it reorders the table: on Gemma 4 E4B, DKV-Mid falls from third to fifth of thirteen operating points, passed by both 8-bit configurations it had outranked; on Gemma 4 E2B and Qwen2.5 7B, DKV-High falls one position, passed by 8-bit StreamingLLM. The eviction and pooling methods are untouched, because they had almost nothing to be charged for. We note that DKV is the representation family for which this correction is largest, and applying it costs that family rank positions in our own tables; it does not, on this suite, push any DKV configuration below the uncompressed dense reference.

6.3 Does task utility predict serving efficiency? No.

Table 4 reports the measured serving track. The clearest evidence is a matched pair. On Gemma 4 E4B at a 4-bit budget, KIVI and SnapKV agree closely on both conventionally reported numbers — 4.55 versus 4.00 bits per element, 61.3% versus 59.3% accuracy — and then diverge completely on the third axis: TTFT of 2.84 s versus 86.04 s (30.3×), peak VRAM of 761 MB versus 2,632 MB (3.46×), hardware multiplier ϕ of 0.99 versus 0.48, and $\mathcal{S}_{\text{sys}}$ of 74.7 versus 59.1. The same pair on Qwen2.5 7B differs by 11.7× in TTFT. Nothing in Axis 1 or Axis 2 anticipates a 30× latency gap.

Across all operating points the association is absent rather than merely weak. Memory saved has no rank relationship to the measured hardware multiplier ($\rho = +0.04$, $p = 0.84$); effective footprint has none to TTFT inflation ($\rho = +0.01$, $p = 0.98$) or to measured peak-VRAM inflation ($\rho = -0.08$, $p = 0.63$). Within each model, algorithmic utility is if anything mildly *anti*-correlated with hardware efficiency ($\rho(\mathcal{S}_{\text{res}}, \phi) = -0.33$, -0.30, -0.49), which is what one expects if the methods that work hardest to preserve quality also do the most work at prefill.

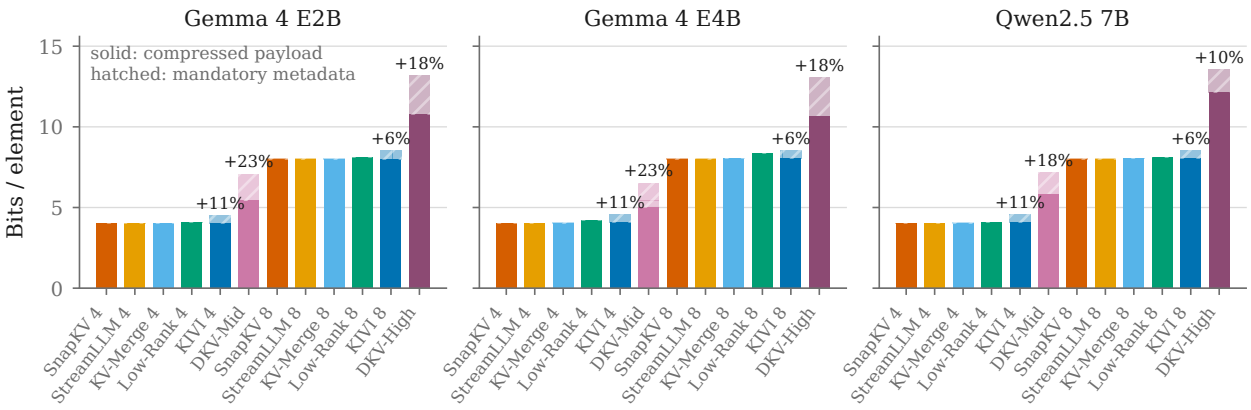


Figure 3: **Nominal payload versus effective bits per element.** Solid bars are the compressed payload conventionally reported as the compression ratio; hatched caps are the mandatory metadata that makes the payload decodable. Labels give the metadata share where it exceeds 5%. Eviction methods carry almost none; grouped quantisation carries 6–11%; factorised representations carry 18–23%.

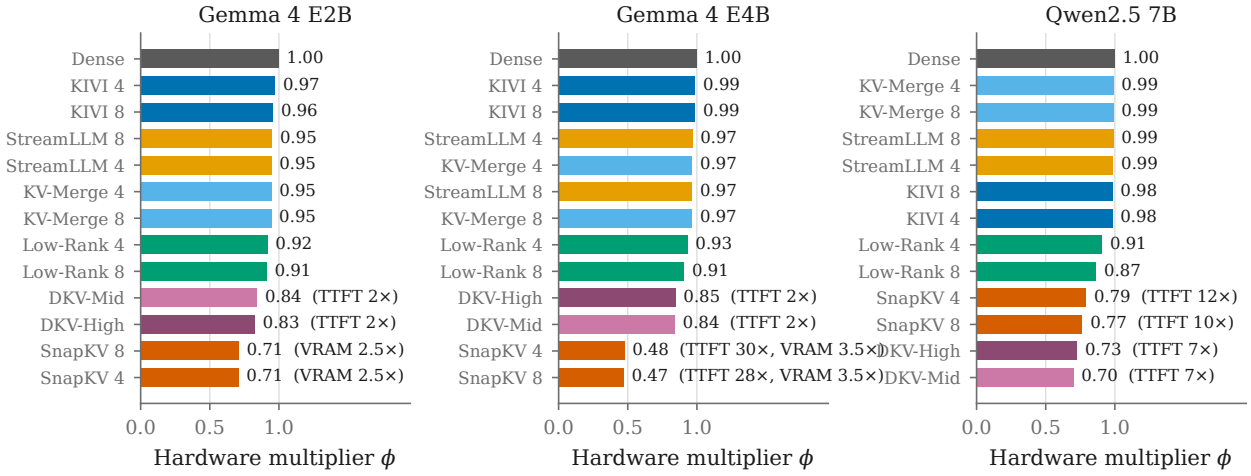


Figure 4: **Where the hardware multiplier is lost.** ϕ for every operating point, annotated with the dominant cause when a method falls materially below the dense reference. The losses are prefill-driven (SnapKV on E4B and 7B, DKV on 7B) or peak-memory-driven (SnapKV on E2B), and in no case are they predictable from the method's position in Figure 3.

Consequently the two tracks rank methods differently. Moving from the Part 1 to the Part 2 ordering shifts a configuration by 1.4, 2.6 and 1.4 positions on average for the three models; the largest single move is SnapKV at 4 bits on Gemma 4 E4B, second of thirteen on algorithmic utility and tenth on serving utility. Because $\mathcal{S}_{\text{sys}}$ embeds $\mathcal{S}_{\text{res}}$ by construction, these shifts are a *lower* bound on the disagreement between the axes.

The penalty is model-dependent, which is itself the point. The same SnapKV integration inflates TTFT by 30.3× on Gemma 4 E4B, 11.8× on Qwen2.5 7B and only 1.04× on Gemma 4 E2B, where its $\phi = 0.71$ is instead driven by a 2.47× peak-VRAM inflation. A single-model systems measurement would have supported a confidently wrong generalisation in either direction.

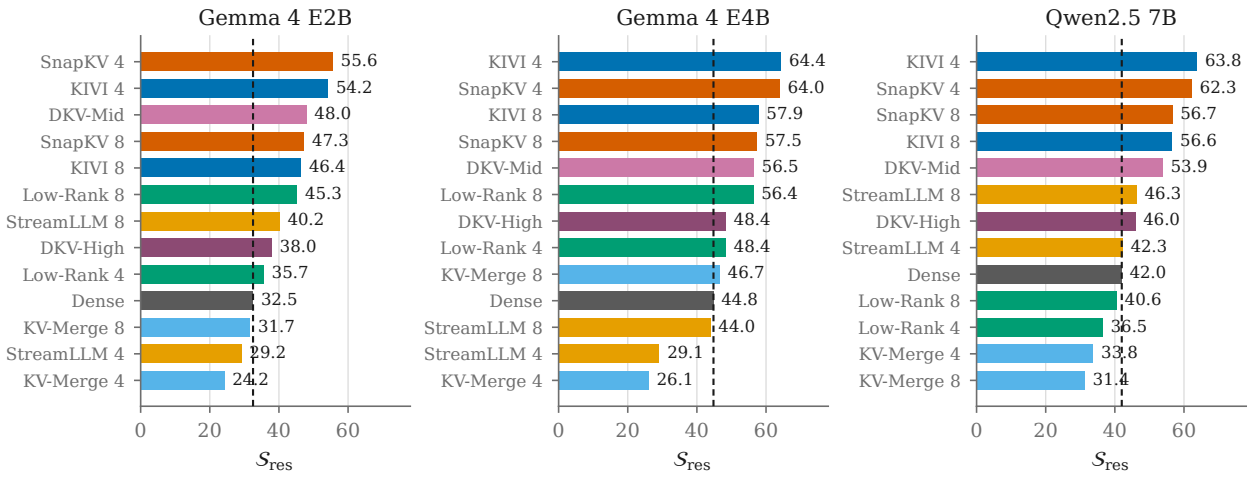


Figure 5: **Part 1 algorithmic utility ($\mathcal{S}_{\text{res}}$) at every operating point,** with the dense reference marked (dashed). Note that dense is not last: at $\alpha = 0.70$, a method that gives up a few accuracy points for a 70% memory saving outscores it.

6.4 Joint view: the two tracks side by side

Figure 5 gives the Part 1 ordering and Figure 4 the hardware term. Read together they support a modest, well-supported summary rather than a universal ranking. On these three models, under these implementations and on this hardware: grouped 4-bit quantisation is the only family that is simultaneously near the top of Axis 2 and essentially free on Axis 3 ($\phi \geq 0.97$ everywhere); eviction with observation pooling is competitive on Axes 1 and 2 and is the most expensive method we measured on Axis 3; pooling and streaming-window methods are cheap on Axis 3 and lose the most on Axis 2; and factorised representations sit in between on all three, with the largest metadata correction. We state these as findings about what we evaluated, not as properties of the method families in general.

6.5 Task structure, and where the limit is the model

Figure 6 breaks accuracy down by task. Two observations bear on how the headline averages should be read.

Variable tracking is a base-model limit, not a compression effect. Every method scores 0.0% on RULER multi-step variable tracking on all three models — including the uncompressed dense reference (a single KIVI configuration on Qwen2.5 7B reaches 3.3%). Whatever is failing here is not the representation. Because this task is one of five, it depresses every reported average by roughly a fifth uniformly; it does not change any ranking, but absolute accuracies in this paper should be read with it in mind.

Multi-needle retrieval is near a scoring floor for the dense reference. Dense accuracy on multi-needle is 32–33% on all three models, consistent with partial credit for locating a subset of the planted needles. It therefore has limited headroom to discriminate between methods. The discriminative signal in our suite comes principally from single-needle retrieval, RULER key–value lookup and multi-hop QA, where dense ranges from 80% to 100% and compressed methods separate cleanly.

6.6 Context scaling and the quality–memory frontier

Figure 7 traces the full 2K–128K ladder on Gemma 4 E2B. Dense accuracy is flat to 32K, falls at 64K and drops sharply at 128K; the better compressed methods track it closely throughout, and the gap between methods does not widen systematically with context. The degradation beyond 64K is therefore shared with the uncompressed baseline and is a property of the model's usable context, not of compression.

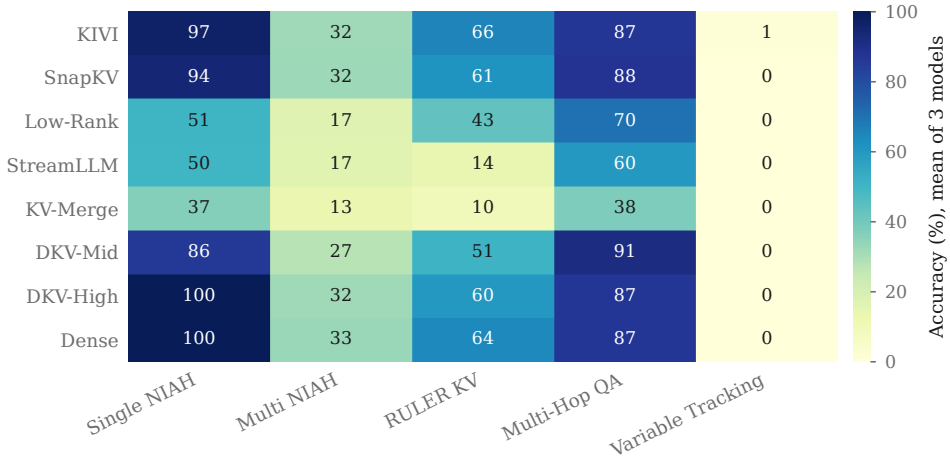


Figure 6: **Accuracy by task,** averaged over the three models and both budget levels. Variable tracking is at zero for every method *and* for the uncompressed dense reference, identifying it as a base-model limitation rather than a compression failure.

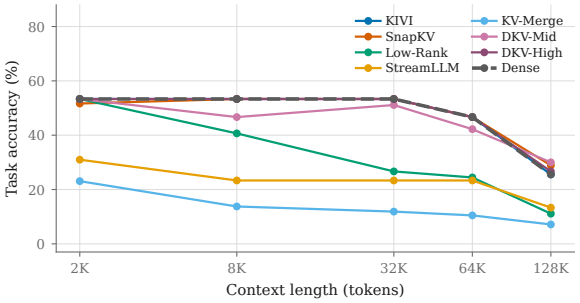


Figure 7: **The 2K–128K ladder on Gemma 4 E2B.** The cliff beyond 64K is shared with the dense reference.

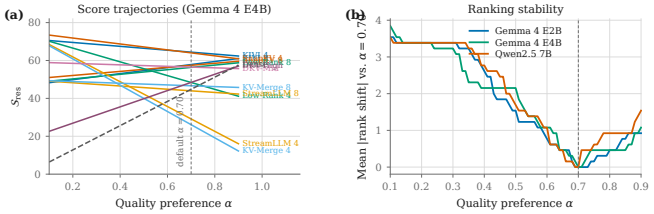


Figure 8: **Sensitivity to α.** Score trajectories and mean rank displacement relative to the default $\alpha = 0.70$.

Figure 9 plots the quality–memory frontier within each model. We compute the frontier per model rather than pooling: accuracy differences between a 2B and a 7B model reflect base-model capability, and a frontier drawn across them would be an artefact. The frontiers are short — on Gemma 4 E2B a single 4-bit configuration dominates everything else, matching dense accuracy at a quarter of the bits — which is itself informative about how much headroom remains at these scales.

7 Discussion

For method authors: report three numbers, not two. The minimum honest summary of a KV-compression method is its effective bits per element *including* metadata, its task accuracy against a paired dense run of the same model, and its measured TTFT, throughput and peak memory relative to that same run. Our results show each of the three carries information the other two do not. Reporting a nominal bit-width understates real footprint by 6–28% in a method-dependent way; reporting a compression ratio without task measurement leaves an 18.7× accuracy range undetermined; reporting both without a systems measurement leaves a 30× latency range undetermined.

For practitioners: the choice depends on which axis binds. If the binding constraint is context length in a fixed memory budget, Axis 1 with honest metadata accounting is the right selector. If it is answer quality, Axis 2 must be measured on the target model, because the ordering of methods changed

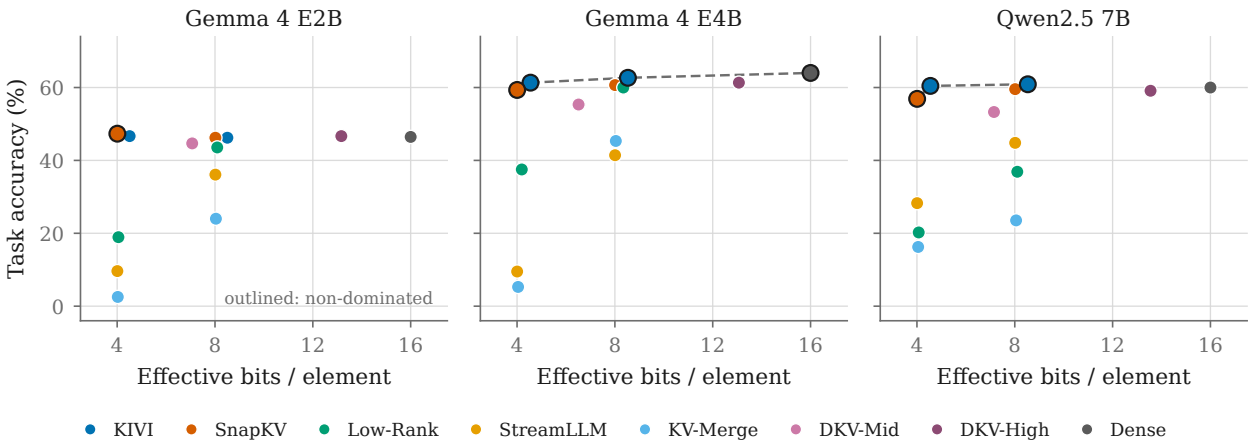


Figure 9: **Quality–memory frontier, computed within each model.** Effective bits include metadata. Outlined markers are non-dominated. Frontiers are not drawn across models, because accuracy differences between models reflect base-model capability rather than the representation.

between our three models. If it is interactive latency or serving cost, only Axis 3 is decisive, and it must be measured on the implementation that will actually be deployed — our own SnapKV and DKV measurements are the clearest illustration of how far an integration can sit from an algorithm's potential.

For benchmark design: keep the tracks apart. It is tempting to collapse everything into one number. We report $\mathcal{S}_{\text{res}}$ and $\mathcal{S}_{\text{sys}}$ separately, and we decline to average them into a headline, because the disagreement between them is the finding. A single score would have concealed that a configuration ranked second on representation utility ranks tenth on serving utility.

An honest note on our own method. One of the evaluated families (DKV) is the authors' own prior work, and it is the family this paper's accounting penalises most. Charging it for metadata moves DKV-Mid from 5.00 to 6.51 effective bits per element on Gemma 4 E4B and costs it two rank positions; DKV-High loses a position on the other two models; and on the serving track the prefill cost of the block compressor leaves both configurations mid-table or lower. DKV remains above the dense reference on Part 1, which we state because it is what the numbers show, but it wins nothing here. We regard this as evidence that the instrument is doing its job rather than as a result to be softened.

8 Scope and Limitations

We state these as constraints on what our evidence supports, not as future work.

1. **Implementation, not algorithm.** Axis 3 measures code paths. Where an upstream fused kernel was unavailable on our platform — SnapKV's patched attention, DKV's paged Triton runtime — our numbers characterise the integration that ran, and must not be read as upper bounds on those algorithms. Table 2 states this per method. This limitation is intrinsic to serving measurement rather than incidental to our setup: there is no implementation-independent serving number.

2. **Analytic versus realised savings.** For KIVI and Low-Rank KV the footprint is analytically accounted while values are retained in a wider dtype, so their Part 1 savings are exact for the scheme but no device memory was freed; their favourable ϕ reflects that the transform is cheap, not that a production integer kernel would be. For SnapKV, StreamingLLM and KV-Merging the cache physically shrinks and both tracks are measured end to end.

3. **Three models, two families, 2B–7B.** All conclusions are scoped to Gemma 4 E2B/E4B and Qwen2.5 7B. Method orderings already differ across these three, which is itself a reason not to extrapolate to 13B+ or 70B+ models, to different attention layouts, or to other tokenisers.

4. **Single-GPU, single-stream.** One RTX 4070 SUPER, batch size one, no tensor or pipeline parallelism, no continuous batching, no paged attention. Multi-tenant serving changes the memory–latency trade-off substantially, and our Axis 3 conclusions do not transfer to it unexamined.

5. **Synthetic, retrieval-weighted tasks.** Our suite probes retrieval, associative lookup and short multi-hop reasoning. It does not cover summarisation, open-ended generation, agentic dialogue or code, where the information a compressed cache must preserve is distributed differently. One task (variable tracking) is at floor for the base models and one (multi-needle) is near a scoring floor for the dense reference, so effectively three of five tasks carry the discriminative signal.

6. **Statistical resolution.** Rank correlations are computed over 12–13 operating points per model and 36 pooled; they are descriptive summaries of a small sample, and we quote p-values only to indicate the absence of a detectable relationship, not to assert one. The matched-budget comparison in §6.1 is a direct observation and does not depend on them. Per-query variance is negligible under greedy decoding (§5), but 75 distinct queries per model is a small suite and per-task accuracies are correspondingly coarse.

7. **The weights α and the ϕ exponents are conventions.** They are reported, varied (Figure 8) and decomposable, but they are not derived from anything. Readers with different deployment priorities should reweight from the component columns.

9 Conclusion

Compression ratio, task accuracy and serving efficiency are three separate properties of a KV-cache compression method, and in our measurements none of them predicts the others. Methods matched to within 0.19 effective bits per element differ by up to 18.7× in task accuracy; methods matched on *both* footprint and accuracy differ by 30× in prefill latency and 3.5× in peak memory; and across 36 operating points the footprint of a representation is statistically uninformative about the hardware efficiency it realises. Charging representations for their mandatory metadata — which the conventional accounting omits — changes reported bitrates by 6–28% and reorders methods, including our own.

CRBench is the instrument these findings required: paired per-query execution against a dense reference, an effective-bits accounting that includes metadata, and two deliberately unmerged tracks. We release the code, the raw per-query records and the scripts that regenerate every number and figure in this paper, so that the accounting choices we argue for can be inspected and disagreed with directly.

Broader Impact Statement

More efficient long-context serving lowers the cost and energy of deploying large models, and makes long-context inference feasible on commodity hardware. It also carries a specific risk that this paper is directly concerned with: lossy KV compression can cause a model to silently omit facts that were present in its context. Our measurements show this failure is not visible in a compression ratio, and that at a fixed bitrate one method can retain 47% of task ability where another retains 2.5%. In retrieval-grounded settings where correctness matters — clinical, legal, financial — a compression method selected on its advertised ratio may quietly drop exactly the evidence the answer depends on. We regard the practice of reporting compression ratios without paired task and systems measurement as the risk worth naming here, and the paired, decoupled reporting we describe as a partial mitigation.

References

Achiam, J., Adler, S., Agarwal, S., et al. (2023). GPT-4 technical report. *arXiv preprint arXiv:2303.08774*.

An, C., Gong, S., Zhong, M., et al. (2023). L-Eval: Instituting standardized evaluation for long context language models. *arXiv preprint arXiv:2307.11088*.

Bai, J., Bai, S., Chu, Y., et al. (2023a). Qwen technical report. *arXiv preprint arXiv:2309.16609*.

Bai, Y., Lv, X., Zhang, J., et al. (2023b). LongBench: A bilingual, multitask benchmark for long context understanding. *arXiv preprint arXiv:2308.14508*.

Bai, Y., Lv, X., Zhang, J., et al. (2024). LongBench v2: Towards deeper understanding of long context large language models. *arXiv preprint arXiv:2412.15204*.

Brown, T., Mann, B., Ryder, N., et al. (2020). Language models are few-shot learners. In *NeurIPS*, vol. 33, pp. 1877–1901.

[Authors withheld for double-blind review] (2026). DKV: Anchor and low-rank differential KV-cache compression for scalable long-context inference. *Preprint under double-blind review*.

DeepSeek-AI (2024). DeepSeek-V2: A strong, economical, and efficient Mixture-of-Experts language model. *arXiv preprint arXiv:2405.04434*.

Dettmers, T., Lewis, M., Belkada, Y., and Zettlemoyer, L. (2022). LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *NeurIPS*, vol. 35, pp. 30318–30332.

Dong, Z., Tang, T., Dai, J., et al. (2023). BAMBOO: A comprehensive benchmark for evaluating long-sequence language models. *arXiv preprint arXiv:2309.13345*.

Ge, S., Zhang, Y., Liu, L., et al. (2023). Model tells you what to discard: Adaptive KV cache compression for LLMs. *arXiv preprint arXiv:2310.01801*.

Hooper, C., Kim, S., Mohammadzadeh, H., et al. (2024). KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *NeurIPS*.

Hsieh, C.-Y., Sun, S., Kriman, S., et al. (2024). RULER: What's the real context size of your long-context language model? *arXiv preprint arXiv:2404.06654*.

Kamradt, G. (2023). Needle in a haystack — pressure testing LLMs. *GitHub repository: gkamradt/LLMTest_NeedleInAHaystack*.

Li, Y., Huang, Y., Yang, B., et al. (2024). SnapKV: LLM knows what you are looking for before generation. In *NeurIPS*.

Liu, Z., Desai, A., Liao, F., et al. (2023). Scissorhands: Exploiting the persistence of importance in LLM KV cache. In *NeurIPS*.

Liu, Z., Yuan, J., Jin, H., et al. (2024). KIVI: A tuning-free asymmetric 2-bit quantization for KV cache. In *ICML*.

Reid, M., Savinov, N., Teplyashin, D., et al. (2024). Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*.

Sharma, P., Ash, J. T., and Misra, D. (2024). The truth is in there: Improving reasoning in language models with layer-selective rank reduction. In *ICLR*.

Touvron, H., Lavril, T., Izacard, G., et al. (2023). LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.

Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention is all you need. In *NeurIPS*, vol. 30, pp. 5998–6008.

Xiao, G., Tian, Y., Chen, B., et al. (2023). Efficient streaming language models with attention sinks. In *ICLR*.

Zhang, Z., Sheng, Y., Zheng, T., et al. (2023). H_2O: Heavy-hitter oracle for efficient generative inference of large language models. In *NeurIPS*.

Zhang, Y., Du, X., Liu, C., et al. (2024a). PyramidKV: Dynamic KV cache compression based on pyramid information funneling. *arXiv preprint arXiv:2406.02069*.

Zhang, X., Chen, Y., Hu, S., et al. (2024b). ∞-Bench: Extending context window to 100K+ for comprehensive evaluation of LLMs. In *ACL*.

Table 3: **Part 1: algorithmic representation utility at every operating point.** b_{pay} is the compressed payload alone — the quantity conventionally reported — and b_{eff} additionally charges mandatory metadata (Definition 3); "meta" is the metadata share of realised footprint. $\mathcal{S}_{\text{res}} = 0.70\,Q_{\text{abs}} + 0.30\,R_{\text{mem}}$ on b_{eff}. Best $\mathcal{S}_{\text{res}}$ per model in **bold**. Paired coverage was 1.00 throughout.

Method	Budget	Accuracy	b_{pay}	b_{eff}	meta	$\mathcal{S}_{\text{res}}$
Gemma 4 E2B (2K–128K)						
KIVI (INT4/INT8)	4	46.7%	4.01	4.51	11.1%	54.2
KIVI (INT4/INT8)	8	46.2%	8.01	8.51	5.9%	46.4
SnapKV	4	47.3%	4.00	4.01	0.2%	**55.6**
SnapKV	8	46.2%	8.00	8.02	0.2%	47.3
Low-Rank KV	4	18.9%	4.05	4.05	0.0%	35.7
Low-Rank KV	8	43.6%	8.09	8.09	0.0%	45.3
StreamingLLM	4	9.6%	4.00	4.01	0.2%	29.2
StreamingLLM	8	36.1%	8.00	8.02	0.2%	40.2
KV-Merging	4	2.5%	4.01	4.02	0.4%	24.2
KV-Merging	8	24.0%	8.01	8.04	0.4%	31.7
DKV-Mid	adaptive	44.7%	5.46	7.07	22.8%	48.0
DKV-High	adaptive	46.7%	10.78	13.17	18.1%	38.0
Dense FP16	–	46.4%	16.00	16.00	0.0%	32.5
Gemma 4 E4B (2K–32K)						
KIVI (INT4/INT8)	4	61.3%	4.05	4.55	10.9%	**64.4**
KIVI (INT4/INT8)	8	62.7%	8.03	8.53	5.8%	57.9
SnapKV	4	59.3%	4.00	4.00	0.1%	64.0
SnapKV	8	60.7%	8.00	8.01	0.1%	57.5
Low-Rank KV	4	37.5%	4.19	4.19	0.0%	48.4
Low-Rank KV	8	60.0%	8.34	8.34	0.0%	56.4
StreamingLLM	4	9.5%	4.00	4.00	0.1%	29.1
StreamingLLM	8	41.4%	8.00	8.01	0.1%	44.0
KV-Merging	4	5.3%	4.03	4.04	0.2%	26.1
KV-Merging	8	45.3%	8.02	8.04	0.2%	46.7
DKV-Mid	adaptive	55.3%	5.00	6.51	23.2%	56.5
DKV-High	adaptive	61.3%	10.68	13.07	18.3%	48.4
Dense FP16	–	64.0%	16.00	16.00	0.0%	44.8
Qwen2.5 7B (2K–32K)						
KIVI (INT4/INT8)	4	60.4%	4.05	4.55	10.9%	**63.8**
KIVI (INT4/INT8)	8	60.9%	8.04	8.53	5.8%	56.6
SnapKV	4	56.9%	4.00	4.01	0.2%	62.3
SnapKV	8	59.6%	8.00	8.02	0.2%	56.7
Low-Rank KV	4	20.2%	4.07	4.07	0.0%	36.5
Low-Rank KV	8	36.9%	8.10	8.10	0.0%	40.6
StreamingLLM	4	28.3%	4.00	4.01	0.2%	42.3
StreamingLLM	8	44.8%	8.00	8.02	0.2%	46.3
KV-Merging	4	16.2%	4.03	4.05	0.4%	33.8
KV-Merging	8	23.5%	8.02	8.05	0.4%	31.4
DKV-Mid	adaptive	53.3%	5.84	7.15	18.3%	53.9
DKV-High	adaptive	59.1%	12.14	13.55	10.4%	46.0
Dense FP16	–	60.0%	16.00	16.00	0.0%	42.0

Table 4: **Part 2: measured serving utility.** TTFT, decode throughput and peak VRAM are descriptive means; "×dense" is the mean per-query ratio against that query's own dense execution. ϕ is computed per query (Equation 7) and averaged. Rows ordered by $\mathcal{S}_{\text{sys}}$ within each model; best per model in **bold**. Serving figures reflect the implementations in Table 2.

Method	**TTFT** (s)	**×dense**	**Decode** (tok/s)	**Peak VRAM** (MB)	ϕ	$\mathcal{S}_{\text{sys}}$
Gemma 4 E2B						
KIVI (INT4/INT8) (4 bpt)	12.51 s	1.00×	12.1	846	0.97	**67.1**
KIVI (INT4/INT8) (8 bpt)	12.51 s	1.00×	11.7	848	0.96	61.2
SnapKV (4 bpt)	13.02 s	1.04×	12.8	2088	0.71	60.2
Low-Rank KV (8 bpt)	12.84 s	1.02×	12.2	851	0.91	59.1
DKV-Mid	22.26 s	1.61×	13.8	1390	0.84	58.9
StreamingLLM (8 bpt)	12.58 s	1.00×	13.7	848	0.95	56.7
SnapKV (8 bpt)	13.06 s	1.04×	13.0	2088	0.71	54.4
Dense FP16	13.18 s	1.00×	13.4	1118	1.00	52.8
Low-Rank KV (4 bpt)	12.76 s	1.02×	12.3	849	0.92	52.5
DKV-High	23.40 s	1.70×	13.8	1391	0.83	51.4
KV-Merging (8 bpt)	12.57 s	1.00×	13.5	848	0.95	50.6
StreamingLLM (4 bpt)	12.58 s	1.00×	13.6	847	0.95	48.8
KV-Merging (4 bpt)	12.60 s	1.00×	14.0	848	0.95	45.3
Gemma 4 E4B						
KIVI (INT4/INT8) (4 bpt)	2.84 s	1.00×	12.8	761	0.99	**74.7**
KIVI (INT4/INT8) (8 bpt)	2.84 s	1.00×	12.8	761	0.99	70.1
Low-Rank KV (8 bpt)	3.25 s	1.14×	13.0	761	0.91	66.7
DKV-Mid	5.06 s	1.65×	12.3	761	0.84	64.8
Low-Rank KV (4 bpt)	3.11 s	1.09×	13.1	761	0.93	61.9
KV-Merging (8 bpt)	2.92 s	1.03×	13.3	762	0.97	61.6
Dense FP16	2.96 s	1.00×	12.4	761	1.00	61.4
StreamingLLM (8 bpt)	2.91 s	1.02×	13.6	762	0.97	59.8
DKV-High	6.25 s	2.03×	12.2	761	0.85	59.3
SnapKV (4 bpt)	86.04 s	30.26×	11.2	2632	0.48	59.1
SnapKV (8 bpt)	80.73 s	28.39×	10.0	2632	0.47	54.4
StreamingLLM (4 bpt)	2.92 s	1.03×	13.6	761	0.97	49.4
KV-Merging (4 bpt)	2.91 s	1.02×	13.7	761	0.97	47.3
Qwen2.5 7B						
KIVI (INT4/INT8) (4 bpt)	5.47 s	1.00×	16.4	3854	0.98	**74.1**
KIVI (INT4/INT8) (8 bpt)	5.46 s	1.00×	16.4	3854	0.98	69.2
SnapKV (4 bpt)	64.08 s	11.77×	24.3	5128	0.79	67.4
SnapKV (8 bpt)	53.30 s	9.79×	20.0	5137	0.77	62.6
StreamingLLM (8 bpt)	5.53 s	1.02×	22.8	3854	0.99	62.1
Dense FP16	5.56 s	1.00×	16.3	3854	1.00	59.4
StreamingLLM (4 bpt)	5.50 s	1.01×	27.1	3854	0.99	59.3
DKV-Mid	39.55 s	6.97×	15.3	3854	0.70	58.8
Low-Rank KV (8 bpt)	7.14 s	1.31×	18.0	3854	0.87	54.4
DKV-High	41.88 s	7.38×	15.3	3854	0.73	54.0
KV-Merging (4 bpt)	5.51 s	1.01×	28.1	3854	0.99	53.4
Low-Rank KV (4 bpt)	6.49 s	1.19×	18.2	3854	0.91	52.8
KV-Merging (8 bpt)	5.52 s	1.01×	23.6	3854	0.99	51.7

A Proof of the Scoring Properties

Proposition 1. Let $\mathcal{S}_{\text{res}}(Q, R) = \alpha Q + (1 - \alpha)R$ with $\alpha \in (0, 1)$. On $[0, 100]^2$, $\mathcal{S}_{\text{res}}$ is strictly increasing in each argument, bounded in $[0, 100]$, and Pareto consistent.

Proof. Monotonicity: $\partial \mathcal{S}_{\text{res}}/\partial Q = \alpha > 0$ and $\partial \mathcal{S}_{\text{res}}/\partial R = 1 - \alpha > 0$. *Boundedness*: $\mathcal{S}_{\text{res}}$ is a convex combination of Q and R, so $\min(Q, R) \leq \mathcal{S}_{\text{res}} \leq \max(Q, R)$, giving $\mathcal{S}_{\text{res}} \in [0, 100]$ with the extremes attained at $(0, 0)$ and $(100, 100)$. *Pareto consistency*: if $Q_A \geq Q_B$ and $R_A \geq R_B$ with at least one strict, then $\mathcal{S}_{\text{res}}(Q_A, R_A) - \mathcal{S}_{\text{res}}(Q_B, R_B) = \alpha(Q_A - Q_B) + (1 - \alpha)(R_A - R_B) > 0$. □

B Alternative Utility Formulations

We considered multiplicative ($Q^\alpha R^{1-\alpha}$), harmonic and threshold-gated alternatives. All satisfy monotonicity and Pareto consistency; the multiplicative and harmonic forms additionally satisfy a substitution-resistance property that the linear form does not, in that a zero on either axis annihilates the score. We chose the linear form regardless, for two reasons. First, annihilation is undesirable here: the dense reference has $R_{\text{mem}} = 0$ by definition and would score zero under any multiplicative form, removing the one anchor every other row is measured against. Second, a linear score can be decomposed back into its inputs by inspection, which matters more for a benchmark intended to be argued with than a marginally better-behaved aggregate would. Figure 8 shows the ranking is stable across α, so the choice of weight is not doing hidden work.

C Per-Model Detail

Tables 3 and 4 in the main text already list every operating point for all three models. Table 5 isolates the matched-budget comparison that supports §6.1, reporting the three models side by side at each nominal budget.

Table 5: **Matched nominal budget across models.** Accuracy per model at each controlled budget, with the mean realised b_{eff}, mean TTFT inflation against the paired dense run, and mean hardware multiplier. Within a block every method holds the same nominal budget, so accuracy differences are attributable to how the budget is spent.

Method	**E2B**	**E4B**	**7B**	b_{eff}	**TTFT ×dense**	ϕ
Nominal budget: 4 bits/element						
KIVI (INT4/INT8)	46.7%	61.3%	60.4%	4.54	1.00×	0.98
SnapKV	47.3%	59.3%	56.9%	4.01	14.36×	0.66
Low-Rank KV	18.9%	37.5%	20.2%	4.11	1.10×	0.92
StreamingLLM	9.6%	9.5%	28.3%	4.01	1.01×	0.97
KV-Merging	2.5%	5.3%	16.2%	4.04	1.01×	0.97
Nominal budget: 8 bits/element						
KIVI (INT4/INT8)	46.2%	62.7%	60.9%	8.52	1.00×	0.98
SnapKV	46.2%	60.7%	59.6%	8.01	13.08×	0.65
Low-Rank KV	43.6%	60.0%	36.9%	8.18	1.16×	0.90
StreamingLLM	36.1%	41.4%	44.8%	8.01	1.01×	0.97
KV-Merging	24.0%	45.3%	23.5%	8.04	1.01×	0.97

D Measurement Protocol

Execution. Prefill is executed in 4,096-token chunks. The context transform is applied to the resident PyTorch KV cache after prefill (or streaming during prefill, for the quantiser), and generation proceeds from

the transformed cache. Decoding is greedy. TTFT is measured with device synchronisation around the full prefill-plus-transform-plus-first-token path, so transform cost is charged to the method that incurs it. Peak device memory is the maximum allocated over the whole query.

What is charged to metadata. Quantisation scales and zero points; SVD factor matrices; DKV anchor and basis state; 32-bit token position indices for eviction methods; and dtype-alignment padding where the scheme requires it. Partial groups that cannot be quantised at the target width are charged at 16 bits.

Reproduction. The scripts under `paper/analysis/` regenerate, in order, every number in the tables (`build_tables.py`), every statistic quoted in §6 (`decoupling_analysis.py`), and every figure (`make_figures.py`), reading only the released per-query records in `results/<model>/progress.jsonl`. A fourth script, `verify_claims.py`, re-derives each quantity quoted in the prose from those outputs and asserts that the manuscript source contains the matching value; it exits non-zero on any drift. No number in this paper is transcribed by hand.